Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
**«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ Инженерный бизнес и менеджмент

КАФЕДРА Промышленная логистика (ИБМ3)

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3
«Простое веб-приложение. Вёрстка»
ПО КУРСУ: РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЙ

НА ТЕМУ:

*Веб-приложение операционного учёта креперии:
меню с технологическими картами блюд*

Выполнил студент ИБМ3-54Б _____ А.К. Рак
(Подпись, дата) (И.О. Фамилия)

Проверил _____ _____
(Подпись, дата) (И.О. Фамилия)

2026 г.

ХОД РАБОТЫ

Задача: знакомство с Node.js и npm. Вёрстка интерфейса с карточками (страница списка с фильтрацией и страница «Подробнее»), данные получать через mock-объекты (коллекция). Добавить кнопку добавления (копирование первой карточки) и кнопку удаления карточки. В хедере на обеих страницах должна быть кнопка «Домой».

Структура проекта

Проект создан как пакет npm: команда `npm init` сформировала файл package.json, библиотека Bootstrap 5 [7] установлена командой `npm install bootstrap` и подключается в index.html из папки node_modules. Скрипты подключены как модули (`type="module"`), поэтому страница открывается через расширение Live Server: при открытии файла напрямую по протоколу file:// браузер блокирует загрузку модулей. Структура проекта показана на рисунке 1:

- `index.html` – единственная HTML-страница с контейнером `#root`, в который отрисовываются страницы приложения;
- `main.js` – точка входа: создаёт главную страницу `MenuPage` и вызывает её отрисовку;
- `mock/` – mock-коллекция позиций меню `crepes`;
- `modules/` – функции расчёта техкарты (`calc.js`) и константы (`consts.js`, порог фудкоста 30 %);
- `pages/` – страницы: `menu` (меню с фильтром) и `tech-card` (техкарта позиции);
- `components/` – компоненты: `header`, `menu-filter`, `crepe-card`, `tech-card`;
- `styles.css` – собственная палитра поверх Bootstrap.

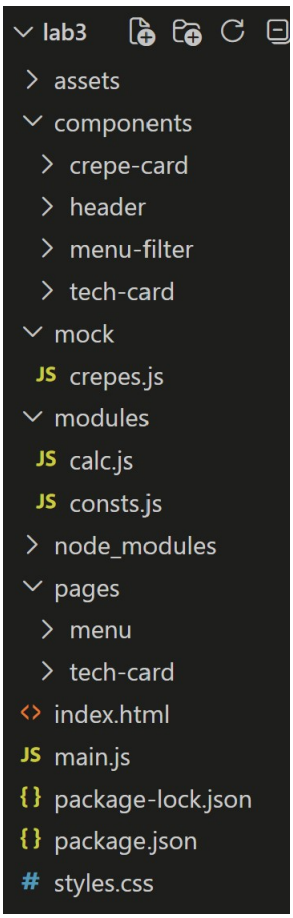


Рисунок 1 – Структура проекта

Имена классов, файлов, переменных, идентификаторов и data-атрибутов отражают тему работы: `crepe`, `techCard`, `ingredient`, `foodCost`, `crepePrice`, `CrepeCardComponent`, `MenuPage`, `TechCardPage`. Общие имена вида product, card или item не используются; исключение составляют только классы самой библиотеки Bootstrap.

Mock-коллекция

Данные приложения хранятся в модуле mock/crepes.js, который экспортирует массив `crepes` из восьми позиций меню: галет, сладких крепов и напитков (рис. 2). Каждая позиция – объект с полями `id`, `code` (номер техкарты), `title`, `category`, `photo`, `crepePrice` (цена продажи), `packaging` (упаковка и её стоимость) и `ingredients` – массивом ингредиентов с массой нетто, процентом потерь и ценой за килограмм. Страницы не хранят данные сами: они импортируют коллекцию и отрисовывают список по её текущему состоянию, поэтому добавление и удаление позиций сразу видно на обеих страницах.

```
export const crepes = [
    {
        id: 1,
        code: "ТК-010",
        title: "Галета Комплет",
        category: "Галеты",
        photo: "assets/crepes/complete.jpg",
        crepePrice: 490,
        packaging: { name: "Крафт-конверт", cost: 9 },
        ingredients: [
            { name: "Тесто гречневое (ТК-001)", net: 90, unit: "кг", pricePerKg: 40, loss: 0 },
            { name: "Ветчина", net: 50, unit: "кг", pricePerKg: 900, loss: 0 },
            { name: "Сыр эмменталь", net: 45, unit: "кг", pricePerKg: 1400, loss: 2 },
            { name: "Яйцо куриное", net: 55, unit: "кг", pricePerKg: 200, loss: 12 },
            { name: "Масло сливочное", net: 5, unit: "кг", pricePerKg: 1050, loss: 0 }
        ]
    },
```

Рисунок 2 – Начало mock-коллекции crepes (mock/crepes.js)

Себестоимость, фудкост и наценка в коллекции не хранятся, а вычисляются функциями модуля modules/calc.js: масса брутто = нетто / (1 − потери), себестоимость = Σ (брутто × цена за кг) + упаковка, фудкост = себестоимость / цена продажи × 100 %, наценка = (цена − себестоимость) / себестоимость × 100 %.

Главная страница меню

Главная страница `MenuPage` (рис. 3) состоит из шапки `HeaderComponent`, заголовка с кнопкой добавления, компонента фильтра `MenuFilterComponent`, строки «Показано N из M» и сетки карточек `CrepeCardComponent`. Метод `render` очищает контейнер `#root`, отрисовывает шапку, вставляет разметку страницы и вызывает метод `renderCrepes`, который строит карточки по коллекции.

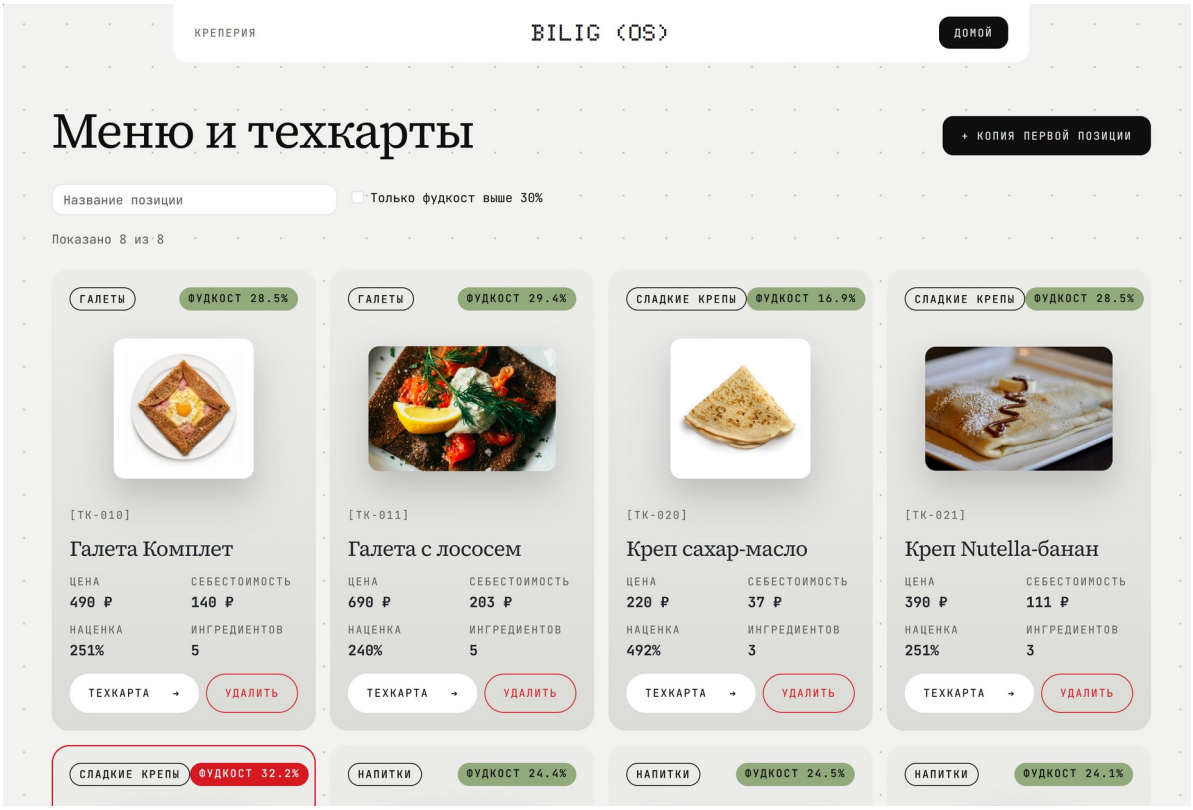


Рисунок 3 – Главная страница меню

Шапка вынесена в отдельный компонент (рис. 4). Он рисует название приложения и кнопку «Домой» и получает обработчик нажатия от страницы, на которой отрисован. На главной странице кнопка «Домой» сбрасывает фильтр и перерисовывает меню, на странице техкарты – возвращает к меню. Поэтому одна и та же шапка используется на обеих страницах.

```
6     getHTML() {
7         return (
8             `
9                 <header class="app-header">
10                    <div class="app-header__side">Креперия</div>
11                    <div class="app-header__logo">BILIG (OS)</div>
12                    <div class="app-header__actions">
13                        <button id="home-button" class="btn btn-dark" type="button">Домой</button>
14                    </div>
15                </header>
16            `
17        )
18    }
```

```
26    render(listener) {
27        this.parent.insertAdjacentHTML('beforeend', this.getHTML())
28        this.addListeners(listener)
29    }
```

Рисунок 4 – Компонент шапки HeaderComponent

Карточка позиции построена на компонентах Bootstrap: карточке `card`, значках `badge` и кнопках `btn`. В верхней части карточки выводятся два значка – категория и фудкост; если фудкост выше порога 30 %, значок и рамка карточки становятся красными. Внизу расположены кнопки «Техкарта» и «Удалить» с атрибутом `data-crepe-id`, по которому обработчик определяет позицию. Обработчики кнопок передаёт страница при отрисовке карточки (рис. 5).

```
42    addListeners(crepe, openListener, deleteListener) {
43        document
44            .getElementById(`open-crepe-${crepe.id}`)
45            .addEventListener('click', openListener)
46        document
47            .getElementById(`delete-crepe-${crepe.id}`)
48            .addEventListener('click', deleteListener)
49    }

51    render(crepe, openListener, deleteListener) {
52        this.parent.insertAdjacentHTML('beforeend', this.getHTML(crepe))
53        this.addListeners(crepe, openListener, deleteListener)
54    }
```

Рисунок 5 – Назначение обработчиков кнопок карточки CrepeCardComponent

Фильтрация списка

Компонент фильтра содержит текстовое поле для отбора по названию и флажок «Только фудкост выше 30 %». При вводе символа (событие `input`) и при изменении флажка (событие `change`) компонент передаёт странице объект с текущими условиями фильтра (рис. 6).

```
22    addListeners(listener) {
23        const titleInput = document.getElementById('crepe-title-filter')
24        const foodCostInput = document.getElementById('crepe-foodcost-filter')
25        const notify = () => {
26            listener({title: titleInput.value, onlyHighFoodCost: foodCostInput.checked})
27        }
28        titleInput.addEventListener('input', notify)
29        foodCostInput.addEventListener('change', notify)
30    }
```

Рисунок 6 – Обработчики полей фильтра MenuFilterComponent

Страница сохраняет условия в поле `menuFilter` и перерисовывает список (рис. 7). Метод `getFilteredCrepes` возвращает новый массив: методом `filter`

в него попадают позиции, название которых содержит введённую строку без учёта регистра, а при отмеченном флажке – только позиции с фудкостом выше порога. Исходная коллекция при фильтрации не изменяется, а счётчик показывает, сколько позиций отобрано из общего числа.

```
35    getFilteredCrepes() {
36        const title = this.menuFilter.title.trim().toLowerCase()
37        return crepes.filter((crepe) => {
38            const titleMatches = crepe.title.toLowerCase().includes(title)
39            const foodCostMatches = !this.menuFilter.onlyHighFoodCost || getFoodCost(crepe) > foodCostLimit
40            return titleMatches && foodCostMatches
41        })
42    }

44    renderCrepes() {
45        this.menuRoot.innerHTML = ''
46        const visibleCrepes = this.getFilteredCrepes()
47        document.getElementById('menu-status').textContent = `Показано ${visibleCrepes.length} из ${crepes.length}`
48        document.getElementById('add-crepe-button').disabled = crepes.length === 0
49
50        visibleCrepes.forEach((crepe) => {
51            const crepeCard = new CrepeCardComponent(this.menuRoot)
52            crepeCard.render(crepe, this.clickOpen.bind(this), this.clickDelete.bind(this))
53        })
54    }

56    changeFilter(menuFilter) {
57        this.menuFilter = menuFilter
58        this.renderCrepes()
59    }
```

Рисунок 7 – Фильтрация и перерисовка списка в MenuPage

При вводе «креп» в списке остаются три сладких крепа (рис. 8), при отмеченном флажке – одна позиция «Креп матча-крем» с фудкостом 32,2 % (рис. 9).

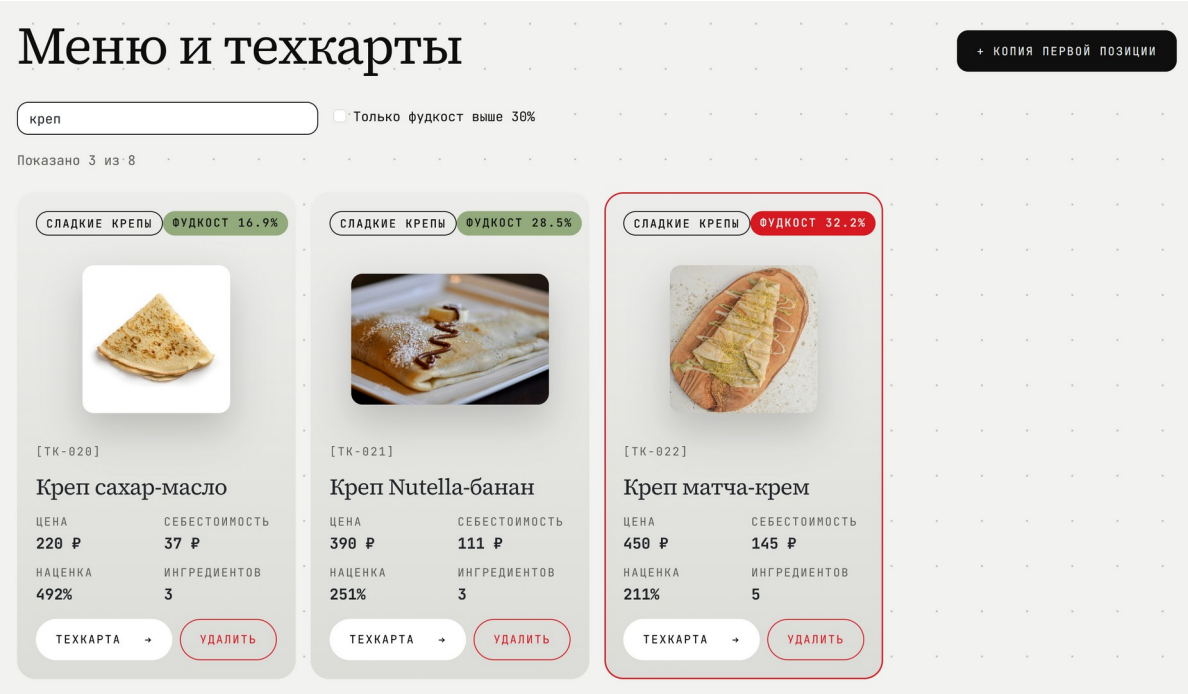


Рисунок 8 – Фильтр по названию «креп»

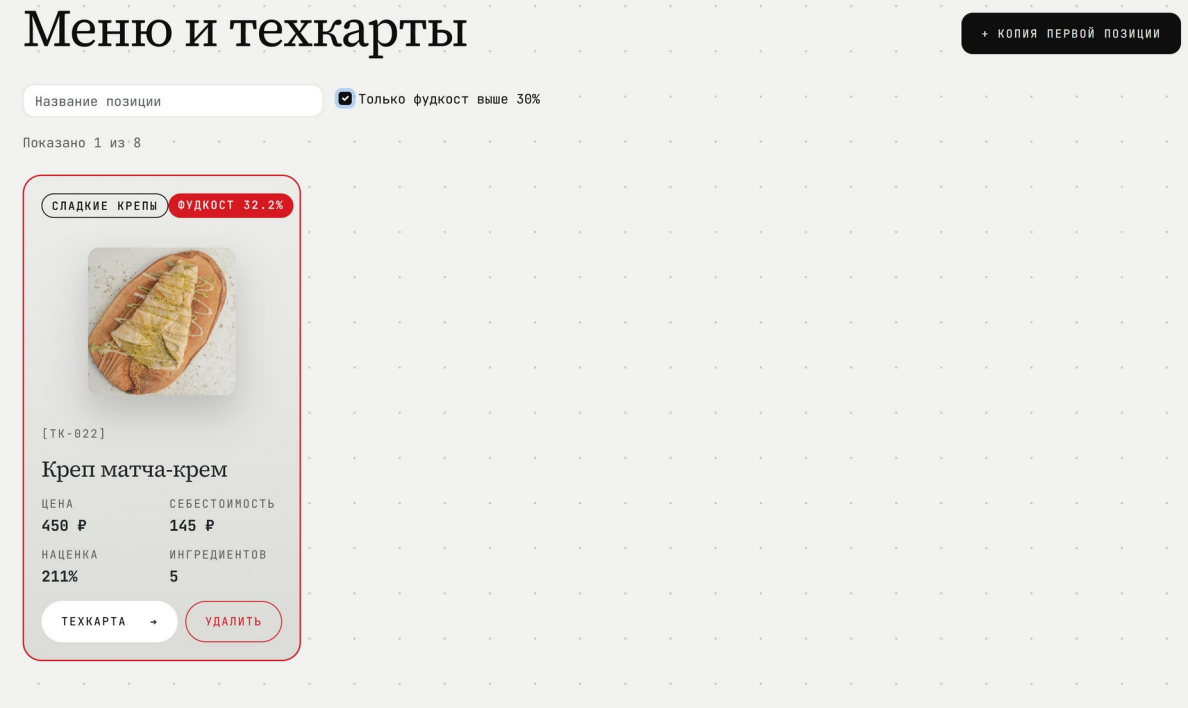


Рисунок 9 – Фильтр по фудкосту выше 30 %

Добавление и удаление позиций

Кнопка «+ Копия первой позиции» вызывает метод `clickAdd` (рис. 10). Он берёт первый элемент коллекции, вычисляет новый `id` как наибольший существующий плюс один и создаёт копию с пометкой «(копия)» в названии. Упаковка и ингредиенты копируются отдельно, чтобы изменения копии не

затрагивали исходную позицию: оператор расширения `...` копирует только верхний уровень объекта. Копия добавляется в коллекцию методом `push`, после чего список перерисовывается.

Кнопка «Удалить» вызывает метод `clickDelete`. Он читает `id` позиции из атрибута `data-crepe-id` нажатой кнопки, находит её индекс в коллекции методом `findIndex`, удаляет элемент методом `splice` и перерисовывает список. Если удалить все позиции, кнопка добавления блокируется, так как копировать нечего.

```
    clickAdd() {
        if (crepes.length === 0) {
            return
        }
        const firstCrepe = crepes[0]
        const newId = Math.max(...crepes.map((crepe) => crepe.id)) + 1
        const crepeCopy = {
            ...firstCrepe,
            id: newId,
            title: `${firstCrepe.title} (копия)`,
            packaging: {...firstCrepe.packaging},
            ingredients: firstCrepe.ingredients.map((ingredient) => ({...ingredient}))
        }
        crepes.push(crepeCopy)
        this.renderCrepes()
    }

    clickDelete(e) {
        const crepeId = Number(e.target.dataset.crepeId)
        const index = crepes.findIndex((crepe) => crepe.id === crepeId)
        if (index !== -1) {
            crepes.splice(index, 1)
        }
        this.renderCrepes()
    }
```

Рисунок 10 – Методы clickAdd и clickDelete

После добавления счётчик меняется с «8 из 8» на «9 из 9», а в списке появляется «Галета Комплет (копия)» (рис. 11). После удаления копии список возвращается к восьми позициям (рис. 12).

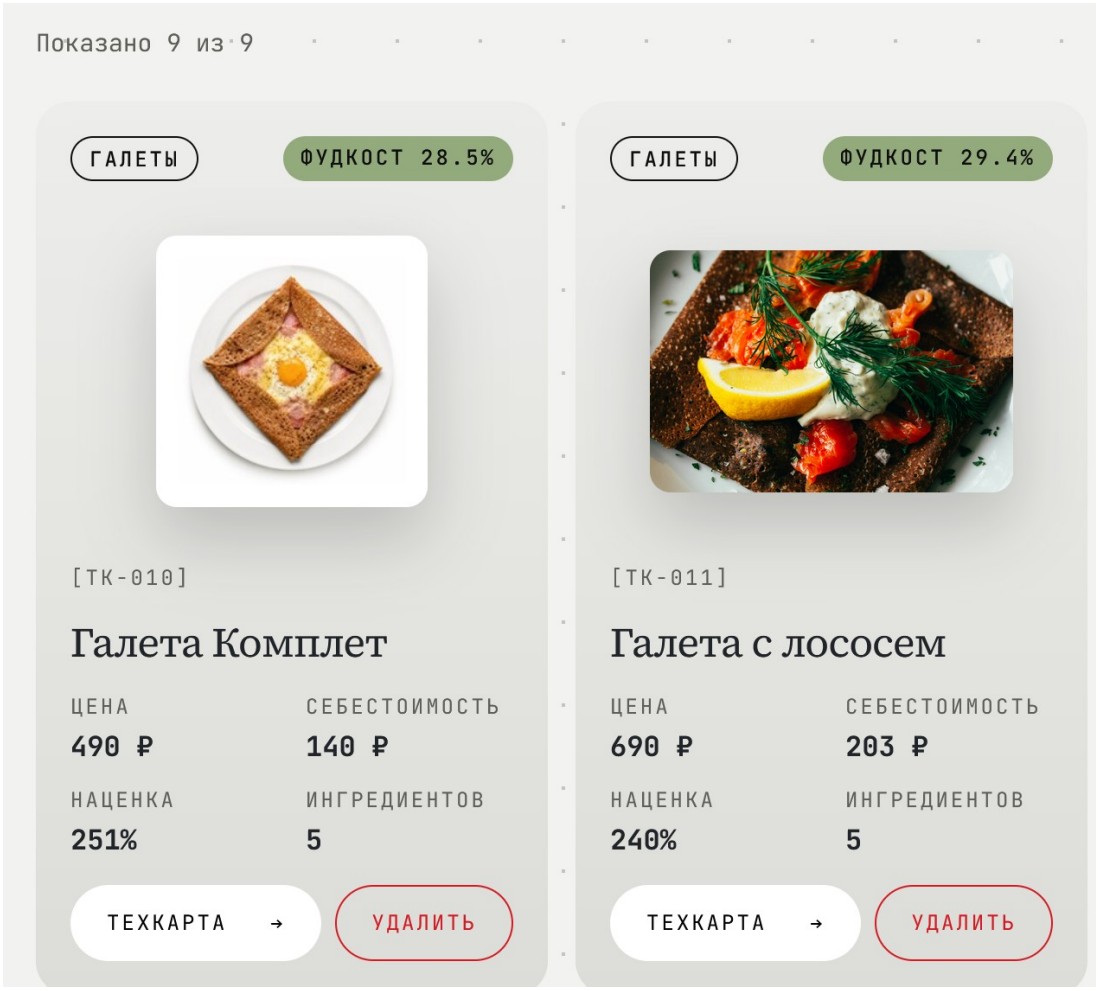


Рисунок 11 – Список после добавления копии первой позиции

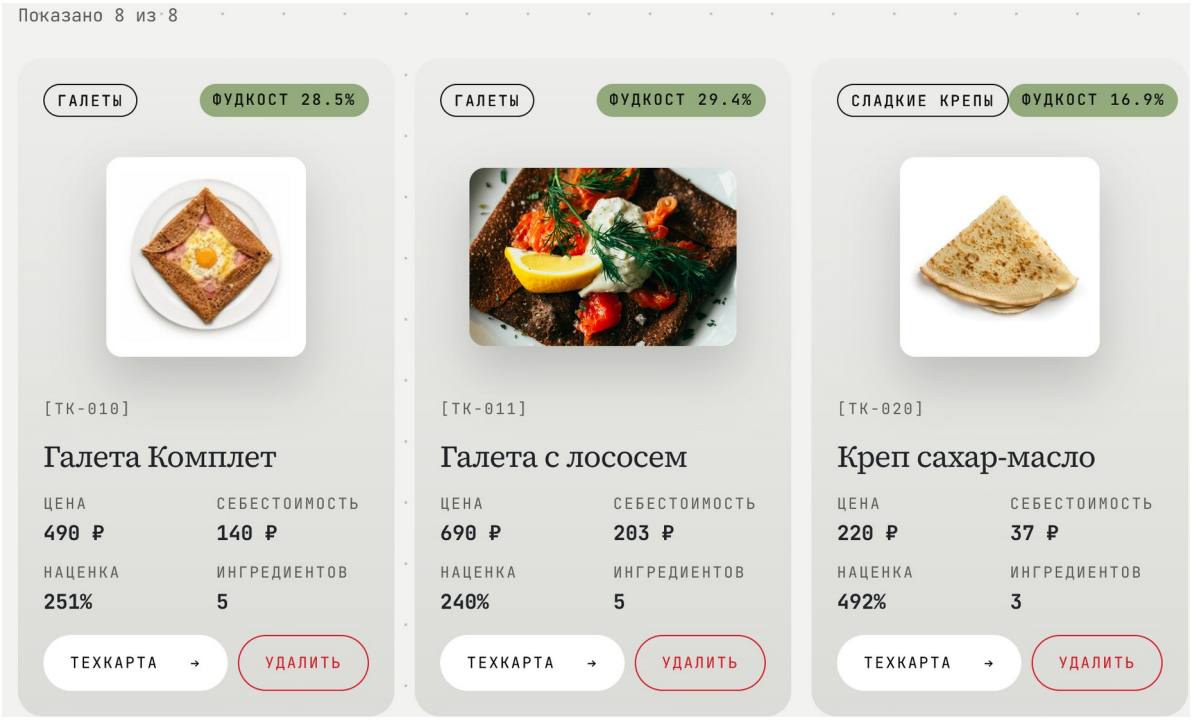


Рисунок 12 – Список после удаления копии

Страница «Подробнее» (техкарта)

По кнопке «Техкарта» страница меню создаёт объект `TechCardPage` и передаёт ему `id` позиции. Страница находит позицию в той же mock-коллекции методом `find`, отрисовывает шапку и компонент `TechCardComponent` (рис. 13). Если позиции нет, например её удалили, вместо пустой страницы выводится сообщение.

```
24      getCrepe() {
25          return crepes.find((menuCrepe) => menuCrepe.id === this.crepeId)
26      }
```

```
33      render() {
34          this.parent.innerHTML = ''
35          const header = new HeaderComponent(this.parent)
36          header.render(this.clickHome.bind(this))
37
38          this.parent.insertAdjacentHTML('beforeend', this.getHTML())
39
40          const crepe = this.getCrepe()
41          if (!crepe) {
42              this.techCardRoot.textContent = 'Позиция не найдена — возможно, её удалили из меню'
43              return
44          }
45          const techCard = new TechCardComponent(this.techCardRoot)
46          techCard.render(crepe)
47      }
```

Рисунок 13 – Страница техкарты TechCardPage

Компонент техкарты показывает фото, цену продажи, себестоимость, фудкост и наценку, а ниже – аккордеон Bootstrap из двух разделов (рис. 14, 15). Раздел «Состав и граммовки» содержит таблицу ингредиентов с массой нетто и брутто, потерями и стоимостью, раздел «Как считается» – формулы расчёта и порог фудкоста.

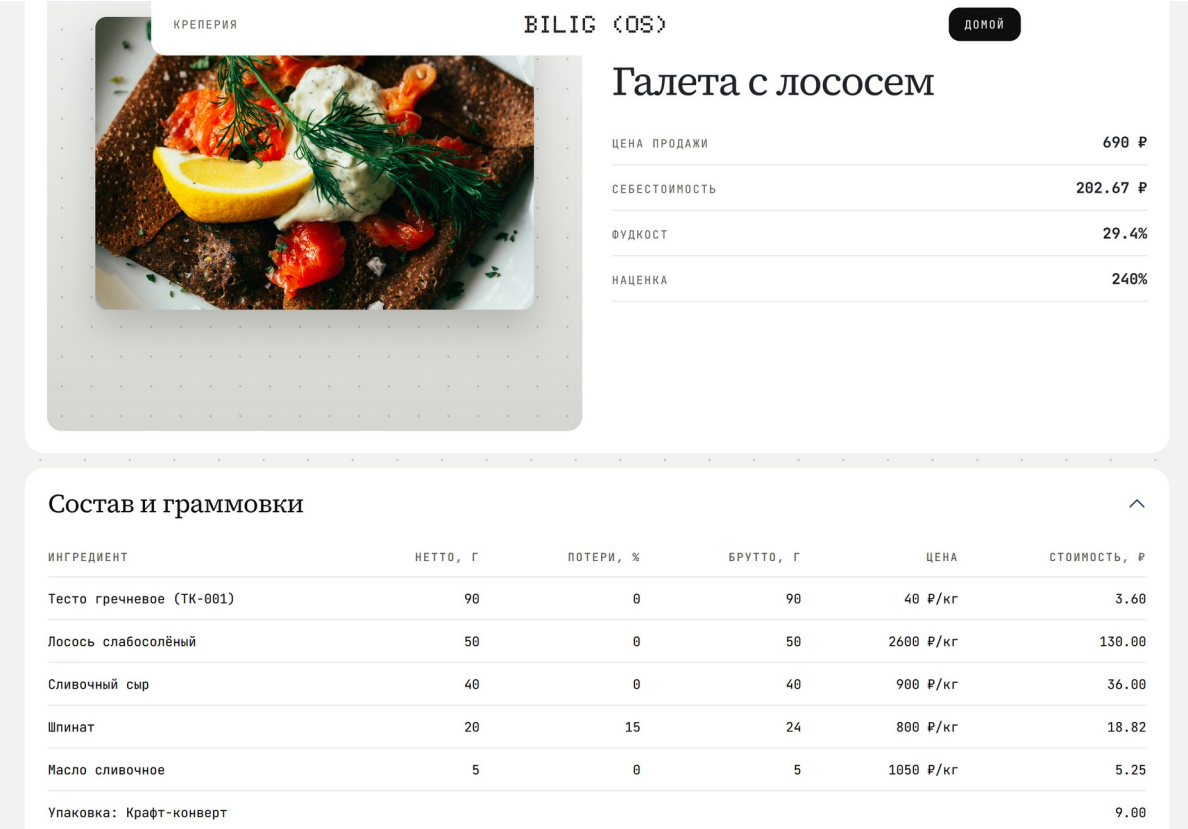


Рисунок 14 – Страница техкарты, раздел «Состав и граммовки»

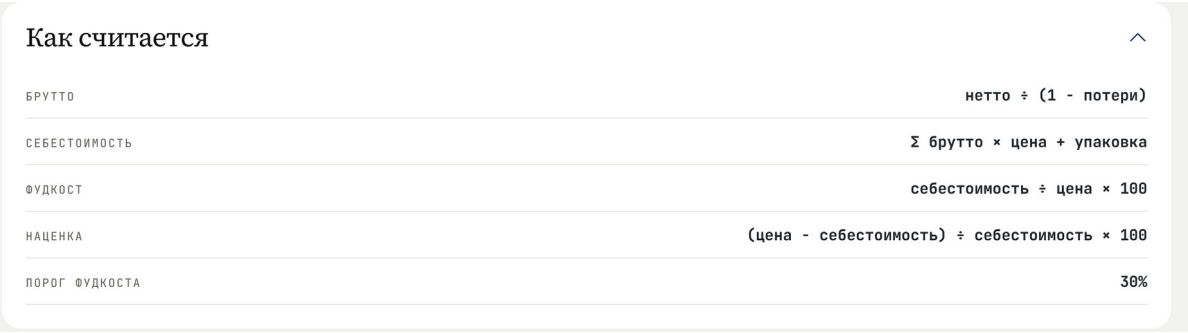


Рисунок 15 – Раздел аккордеона «Как считается»

Собственная палитра поверх Bootstrap

Стандартная палитра Bootstrap заменена собственной (рис. 16). Цвета объявлены переменными в селекторе `:root`: почти чёрный #0F0F0F для текста и кнопок, светло-серый #F2F2F0 для фона страницы, белый #FFFFFF для карточек, оливково-зелёный #93AB7C для наведения и красный #D71921 для превышения фудкоста. Переменные самого Bootstrap (`--bs-success-rgb`, `--bs-danger-rgb`, `--bs-btn-bg` и др.) переопределены этими значениями, поэтому стандартные классы `btn-dark`, `text-bg-success` и `text-bg-danger` окрашиваются в цвета темы без изменения разметки.

```css
:root {
    --ink: #0f0f0f;
    --paper: #f2f2f0;
    --panel: #ffffff;
    --dot: #c4c4bf;
    --muted: #5f5f5a;
    --line: #e6e6e2;
    --accent: #93ab7c;
    --red: #d71921;
    --bs-secondary-rgb: 15, 15, 15;
    --bs-success-rgb: 147, 171, 124;
    --bs-danger-rgb: 215, 25, 33;
}
```

```css
.btn-primary, .btn-dark {
    --bs-btn-bg: var(--ink);
    --bs-btn-border-color: var(--ink);
    --bs-btn-color: var(--paper);
    --bs-btn-hover-bg: var(--accent);
    --bs-btn-hover-border-color: var(--accent);
    --bs-btn-hover-color: var(--ink);
    --bs-btn-active-bg: var(--accent);
    --bs-btn-active-border-color: var(--accent);
    --bs-btn-active-color: var(--ink);
}
```

Рисунок 16 – Палитра и переопределение переменных Bootstrap в styles.css

Исправление ошибок методических указаний

Ошибки в примерах кода методических указаний и способы их исправления приведены в таблице 1.

Таблица 1 – Исправление ошибок методических указаний

№	Ошибка в методических указаниях	Исправление
1	В разметке главной страницы закрывающий тег записан как вместо	Во всех шаблонах используются парные закрывающие теги
2	Лишняя кавычка в атрибутах кнопки карточки	Атрибуты кнопок записаны без лишних символов: id="open-crepe-${crepe.id}" data-crepe-id="${crepe.id}"
3	На странице товара не импортирован компонент	Каждая страница явно импортирует все используемые компоненты

№	Ошибка в методических указаниях	Исправление
	карточки	(pages/menu, pages/tech-card)
4	Модули с type="module" не работают при открытии index.html двойным щелчком	Страница открывается через Live Server; по протоколу file:// браузер блокирует загрузку модулей

GET-запрос получения HTML-страницы

При открытии приложения браузер отправляет серверу Live Server GET-запрос за документом index.html. Во вкладке Network инструментов разработчика (F12) выбран этот запрос, на вкладке Response отображается полученный HTML-код: подключение стилей Bootstrap, контейнер `#root` и модуль main.js (рис. 17). Карточек меню в этом ответе нет: их строит JavaScript уже в браузере.

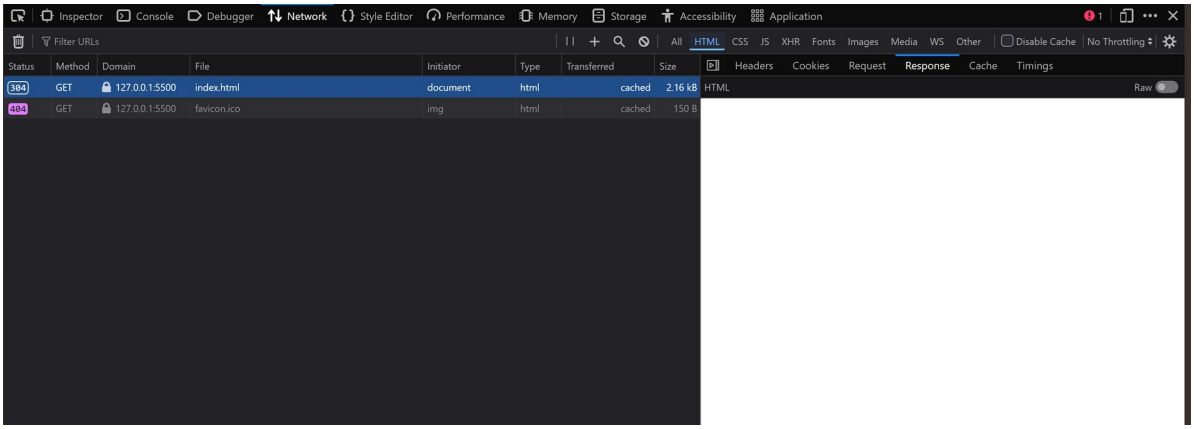


Рисунок 17 – GET-запрос index.html во вкладке Network

Ответы на контрольные вопросы

Структура проекта. Проект делится на точку монтирования index.html, точку входа main.js, страницы (pages), компоненты (components), данные (mock), вспомогательные модули (modules) и файлы npm (package.json, node_modules). Страница собирается из компонентов, компоненты получают данные и обработчики от страницы.

Что такое MVC. Model–View–Controller – способ разделить приложение на три части: модель хранит данные, представление показывает их пользователю,

контроллер обрабатывает действия пользователя и изменяет модель. В работе модель – mock-коллекция crepes, представление – страницы и компоненты, формирующие разметку, контроллер – обработчики удаления (`clickDelete`), копирования (`clickAdd`) и фильтра (`changeFilter`), которые меняют коллекцию или условия отбора и вызывают перерисовку.

Что такое роутинг. Роутинг – выбор страницы, которую нужно показать, в зависимости от действия пользователя или адреса. В приложении используется простейший роутинг без изменения адреса: при переходе текущая страница очищает контейнер `#root` и создаёт объект другой страницы (`MenuPage` → `TechCardPage` по кнопке «Техкарта», обратно – по кнопке «Домой»). Адрес в браузере не меняется, поэтому кнопка «Назад» браузера и прямая ссылка на техкарту не работают; в полноценных приложениях для этого используют History API или хеш в адресе.

Ключевые этапы рендера. Браузер загружает index.html по GET-запросу, строит DOM и подключает стили; затем загружает модуль main.js и его импорты; main.js создаёт `MenuPage` и вызывает `render`; `render` очищает `#root`, формирует HTML-строки компонентов и вставляет их методом `insertAdjacentHTML`; после вставки компоненты находят свои кнопки по `id` и назначают обработчики. При действии пользователя обработчик меняет данные и повторяет отрисовку только нужной части – `renderCrepes` перерисовывает сетку карточек, не трогая шапку и поле фильтра.

Выполнение заданий

Выполнение требований к лабораторной работе приведено в таблице 2.

Таблица 2 – Выполнение заданий лабораторной работы № 3

№	Требование	Реализация
1	Данные из mock-коллекции в отдельном модуле	Массив crepes в mock/crepes.js; страницы импортируют коллекцию
2	Страница списка с фильтрацией	Фильтр по названию и по фудкосту, список перерисовывается по коллекции (components/menu-filter,

№	Требование	Реализация
		pages/menu)
3	Кнопка добавления: копия первой карточки с новым id	Метод clickAdd в pages/menu
4	Кнопка удаления на каждой карточке	Кнопка «Удалить» в components/crepe-card, метод clickDelete в pages/menu
5	Хедер с кнопкой «Домой» на обеих страницах	HeaderComponent в components/header
6	Страница «Подробнее» по клику на карточку	TechCardPage и TechCardComponent (pages/tech-card, components/tech-card)
7	Компоненты Bootstrap по теме	Карточки card, значки badge, аккордеон accordion, кнопки, поле ввода и флажок, таблица
8	Свои цвета поверх Bootstrap	CSS-переменные и переопределение переменных --bs-* в styles.css
9	Исправление ошибок методических указаний	Таблица 1
10	Тематические имена	crepe, techCard, ingredient, foodCost, crepePrice во всех файлах проекта

ВЫВОД

В ходе работы освоены платформа Node.js и менеджер пакетов npm, подключение библиотеки Bootstrap через npm и разбиение интерфейса на страницы и компоненты на классах JavaScript с модулями import/export. Создано двухстраничное приложение «меню – техкарта» для креперии: данные берутся из mock-коллекции crepes, список фильтруется по названию и по фудкосту, позиции добавляются копированием первой и удаляются из коллекции с перерисовкой списка, шапка с кнопкой «Домой» вынесена в общий компонент и выводится на обеих страницах. Использованы карточки, значки и аккордеон Bootstrap в собственной палитре, исправлены ошибки методических указаний, имена в коде приведены к теме работы.

Исходный код размещён в репозитории GitHub: https://github.com/rakulasi/webchi-labs, ветка lab3-creperie-tech-cards.

СПИСОК ЛИТЕРАТУРЫ

1. Курс «JavaScript»: лекции и методические указания [Электронный ресурс] // GitHub. – URL: https://github.com/iu5git/JavaScript (дата обращения: 28.10.2026).
2. MDN Web Docs. Справочник по HTML, CSS и JavaScript [Электронный ресурс]. – URL: https://developer.mozilla.org/ru/ (дата обращения: 28.10.2026).
3. Кантор И. Современный учебник JavaScript [Электронный ресурс]. – URL: https://learn.javascript.ru (дата обращения: 28.10.2026).
4. HTMLBook. Справочник по HTML и CSS [Электронный ресурс]. – URL: http://htmlbook.ru (дата обращения: 28.10.2026).
5. Google Fonts: Montserrat, Jost [Электронный ресурс]. – URL: https://fonts.google.com (дата обращения: 28.10.2026).
6. Breizh Café. Crêperie bretonne [Электронный ресурс]. – URL: https://www.breizhcafe.com (дата обращения: 28.10.2026).
7. Bootstrap 5.3. Документация [Электронный ресурс]. – URL: https://getbootstrap.com/docs/5.3/ (дата обращения: 28.10.2026).
8. ВКонтакте для разработчиков. Справочник методов API [Электронный ресурс]. – URL: https://dev.vk.com/ru/reference (дата обращения: 28.10.2026).